IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Phase 1: Brainstorming & Ideation Phase

Objective: Identify the need for client-side form behaviour and define how Client Scripts and UI Policies can improve the Incident form in ServiceNow.

1. Problem Identified

- Incident forms may display fields that are not relevant to the selected situation.
- Users may enter incomplete or invalid information when required fields are not clearly enforced.
- Form behaviour should respond immediately to user actions without unnecessary page reloads.
- The solution must make the Incident form easier to use while preserving accurate records.

2. Proposed Idea

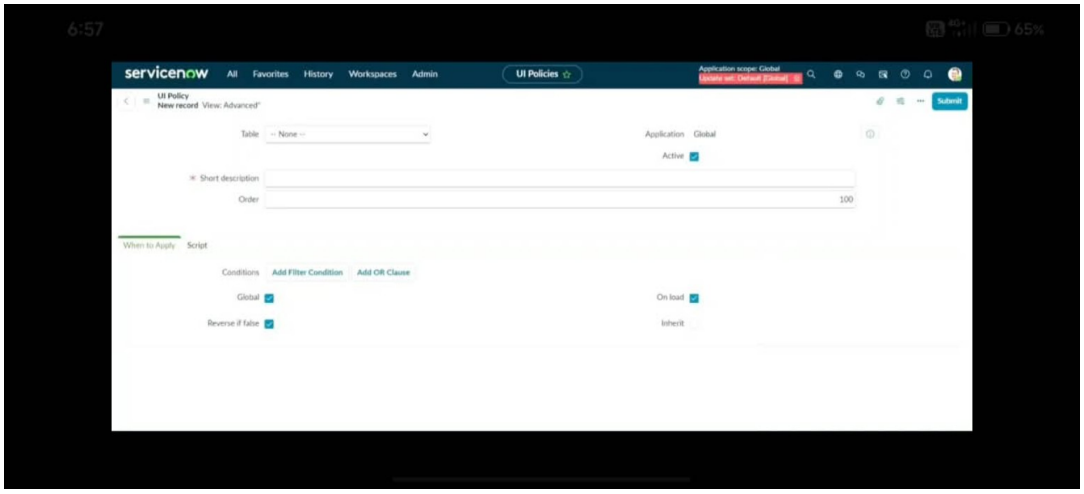
- Configure Client Scripts to control form behaviour using client-side JavaScript.
- Configure UI Policies to make fields mandatory, visible, or read-only based on conditions.
- Apply the rules to the Incident table and test them against relevant field values.
- Keep the configuration simple, consistent, and easy to maintain.

3. Initial Implementation Idea

- Use an appropriate Client Script type, such as onLoad or onChange, for the required behaviour.
- Create a UI Policy with clear conditions and configure its UI Policy Actions.
- Use scripts only where declarative UI Policy settings are insufficient.
- Test normal, alternate, and empty-value scenarios to confirm the expected behaviour.

4. Expected Benefit

- Improve data quality and consistency in Incident records.
- Reduce user errors by guiding users while completing the form.
- Make relevant fields visible, mandatory, or read-only when needed.
- Provide a clearer and more efficient ServiceNow user experience.



IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Phase 2: Requirement Analysis Phase

Objective: Define the functional requirements and conditions for applying a UI Policy to Incident records based on the Impact field.

1. Functional Requirements

- The solution must work on the Incident table in ServiceNow.
- The UI Policy condition must check whether Impact is set to “1 - High”.
- When the condition is true, the Assignment group field must be mandatory.
- The policy should respond when the form loads and when the condition changes, as configured.

2. UI Policy Conditions

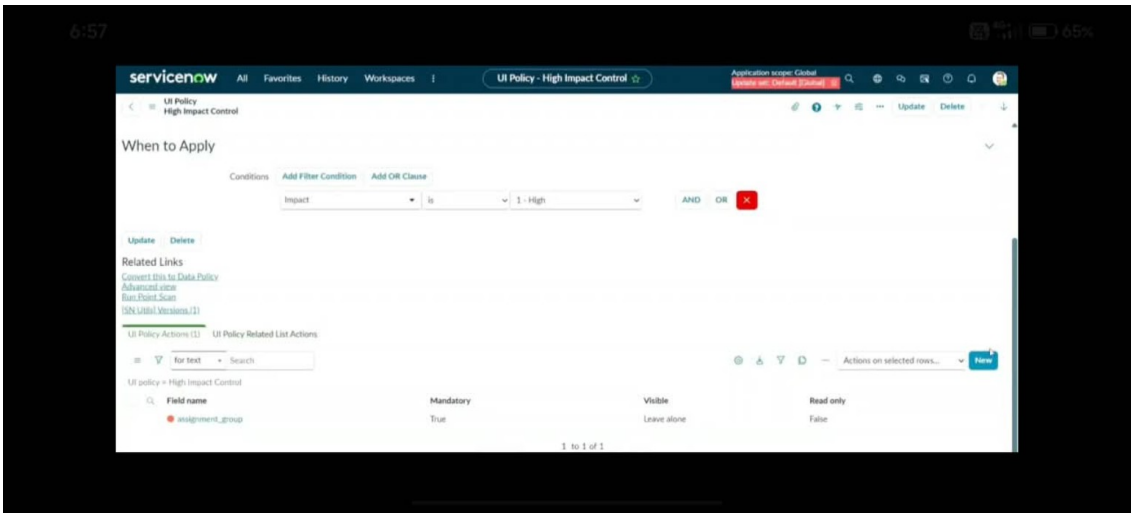
- Create a UI Policy named “High Impact Control”.
- Set the condition to: Impact is 1 - High.
- Keep the policy active so it can be evaluated on the Incident form.
- Configure the policy action for the assignment_group field.

3. Expected Field Behaviour

- When Impact is High, Assignment group becomes mandatory.
- The Visible setting is left unchanged, so the policy does not change field visibility.
- The Read-only setting remains false, so the field remains editable.
- When the condition is not met, the reverse-if-false setting determines whether the mandatory rule is reverted.

4. Validation and Acceptance Criteria

- Open an Incident and select Impact = 1 - High; confirm Assignment group is mandatory.
- Change Impact to another value and verify the field behaviour follows the policy configuration.
- Submit an Incident with High impact and no Assignment group; the form should require the field.
- Confirm that unrelated fields and Incident form behaviour remain unchanged.



IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Phase 3: Project Design Phase

Objective: Design the Incident form behaviour and define how the Client Script and UI Policy will work together to improve data entry and enforce the required field conditions.

1. Form Design

- Use the standard ServiceNow Incident form to capture caller, category, subcategory, impact, urgency, assignment group, assigned to, short description, and description.
- Keep the form layout clear so users can identify the fields required to create an Incident.
- Use the Impact field as the condition that controls the Assignment group requirement.

2. UI Policy Design

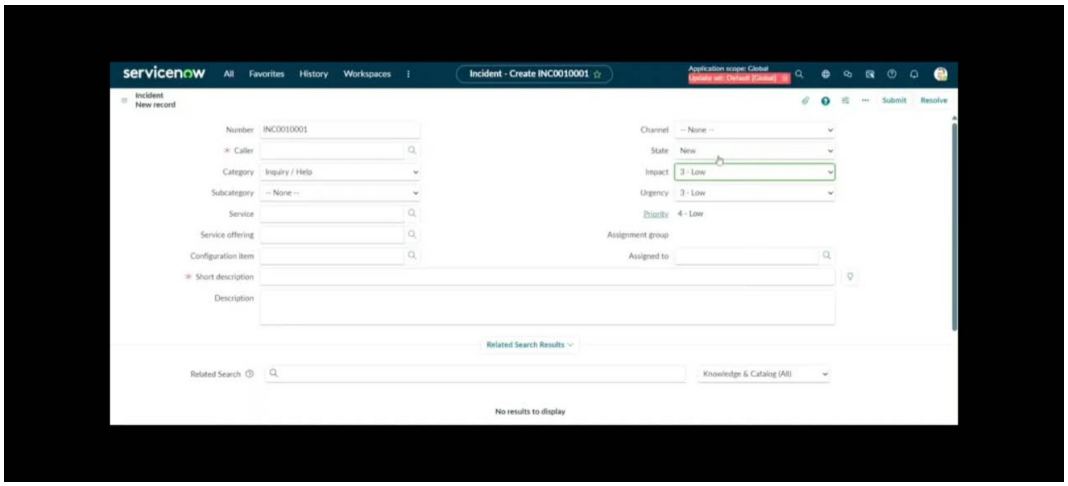
- Create an active UI Policy named “High Impact Control” on the Incident table.
- Set the condition to: Impact is 1 - High.
- Add a UI Policy Action for the assignment_group field and set Mandatory to true.
- Leave Visible unchanged and Read-only set to false, as shown in the policy configuration.

3. Client Script Design

- Use a Client Script only for behaviour that cannot be handled clearly through UI Policy settings.
- Choose the appropriate script type, such as onLoad or onChange, based on when the form needs to respond.
- Keep client-side logic simple and avoid duplicating the same rule in both the script and UI Policy.

4. Design Validation

- When Impact is High, verify that Assignment group becomes mandatory.
- When Impact is Low or another value, verify that the field behaviour follows the UI Policy’s reverse-if-false configuration.
- Confirm the form remains editable and other fields are not unintentionally changed.
- Test the design before moving to implementation and final testing.



IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Phase 4: Project Planning Phase

Objective: Plan the configuration, implementation, and testing tasks for the Incident Client Script and UI Policy in ServiceNow.

1. Project Tasks

- Review the requirements for the Incident form and the High Impact Control UI Policy.
- Create and configure the UI Policy on the Incident table.
- Create the Client Script to handle the required onChange behaviour for the Impact field.
- Test the form behaviour for High, Medium, and Low impact values.

2. Implementation Plan

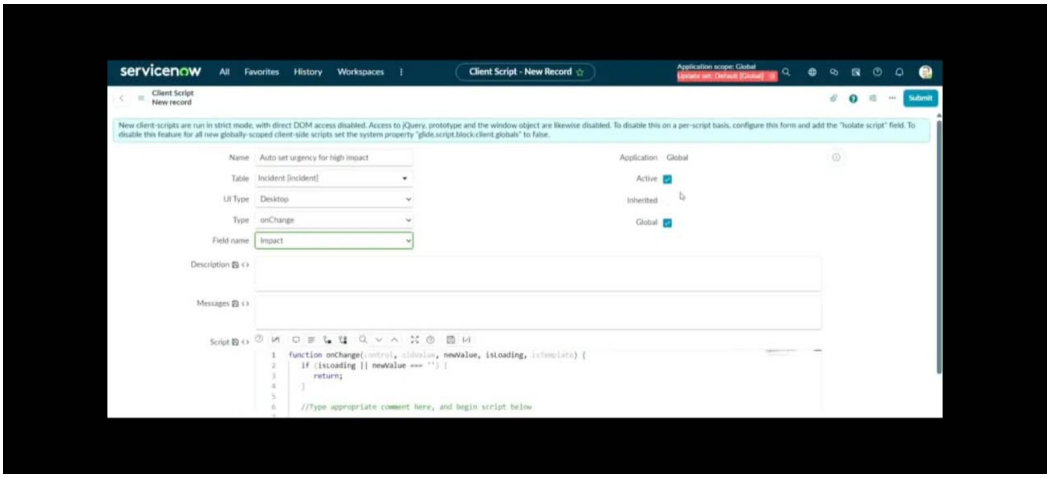
- Configure the UI Policy condition: Impact is 1 - High.
- Set the Assignment group field to mandatory when the condition is true.
- Configure the Client Script named “Auto set urgency for high impact” with Type set to onChange and Field name set to Impact, as shown in the screenshot.
- Add and validate the required script logic without duplicating UI Policy behaviour.

3. Testing Plan

- Change Impact to High and confirm the expected urgency and mandatory-field behaviour.
- Change Impact to Medium or Low and confirm the form responds correctly.
- Check the behaviour when the form first loads and when field values are changed.
- Confirm there are no script errors and unrelated Incident fields work normally.

4. Deliverables and Timeline

- Deliverables: configured UI Policy, Client Script, test results, and screenshots.
- Complete configuration before final validation.
- Record any issues found during testing and correct them before submission.
- Prepare the final project documentation with each phase and its corresponding screenshot.



IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Phase 5: Project Development Phase

Objective: Develop and configure the Client Script and UI Policy for the ServiceNow Incident form to enforce the required behaviour for high-impact incidents.

1. Client Script Configuration

- Create a Client Script on the Incident table in ServiceNow.
- Set the UI Type to Desktop and choose the appropriate script type for the required action.
- Use an onChange Client Script on the Impact field to respond when the Impact value changes.
- Keep the script active and add clear comments describing its purpose.

2. UI Policy Configuration

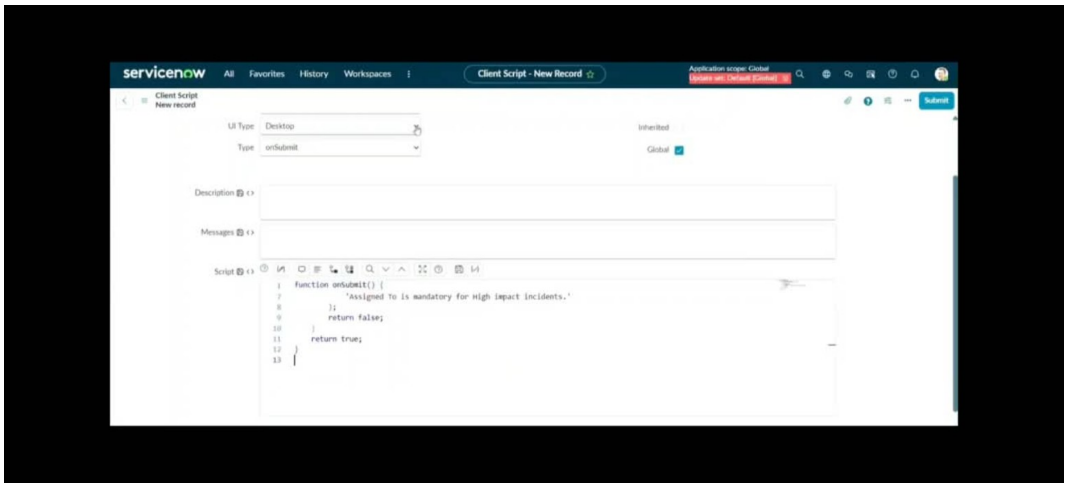
- Create an active UI Policy named “High Impact Control”.
- Set the condition to Impact is 1 - High.
- Add a UI Policy Action for Assignment group and make the field mandatory when the condition is true.
- Leave field visibility unchanged and keep the field editable.

3. Form Submission Validation

- Use an onSubmit Client Script to validate required information before an Incident is submitted.
- If the required assignment information is missing for a high-impact Incident, display a clear message and prevent submission.
- Allow submission when the validation condition is satisfied.
- Ensure the validation logic matches the field requirement configured in the UI Policy.

4. Development Checks

- Test the script with High, Medium, and Low Impact values.
- Confirm the correct message appears when required information is missing.
- Verify valid Incident records can be submitted successfully.
- Review the browser console and form behaviour for errors before moving to testing.



IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Phase 6: Project Testing Phase

Objective: Test the Client Script and UI Policy on the ServiceNow Incident form to verify that field behaviour and state-change validation work as intended.

1. Test Environment

- Use the ServiceNow Incident table in the development or test instance.
- Ensure the Client Script “Prevent state change via list edit” is active and configured for the State field.
- Use test Incident records so testing does not affect live operational data.

2. Test Cases

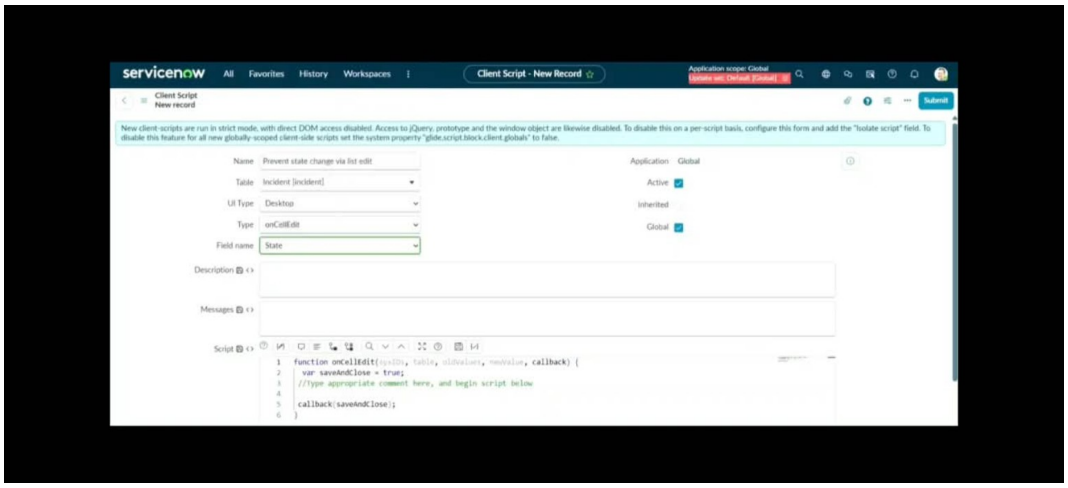
- Attempt to edit the State field through list editing and verify the script behaviour.
- Edit the State field through the Incident form and confirm normal form behaviour is preserved.
- Test the onCellEdit callback and verify that the save is accepted or rejected according to the intended rule.
- Repeat with different Incident states and record the result for each case.

3. Expected Results

- The script should prevent disallowed state changes through list editing when the validation rule is triggered.
- Allowed state changes should save successfully.
- The user should receive clear feedback when a change is rejected, if a message is configured.
- No unrelated Incident fields or records should be affected.

4. Test Outcome and Fixes

- Record each test case as Pass or Fail.
- Correct any issues in the Client Script and repeat the failed test cases.
- Check for JavaScript errors and confirm the callback returns the intended result.
- Retest the UI Policy to ensure the Assignment group requirement for High Impact remains correct.



IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Phase 7: Project Documentation Phase

Objective: Prepare clear project documentation for the ServiceNow Incident Client Script and UI Policy, including configuration steps, screenshots, test results, and the final outcome for future reference.

1. Documentation Components

- Project overview, objective, and identified requirements.
- UI Policy conditions and field actions for high-impact Incidents.
- Client Script configuration, script type, target field, and logic.
- Test scenarios, expected behaviour, screenshots, and final results.

2. Documenting UI Policy Configuration

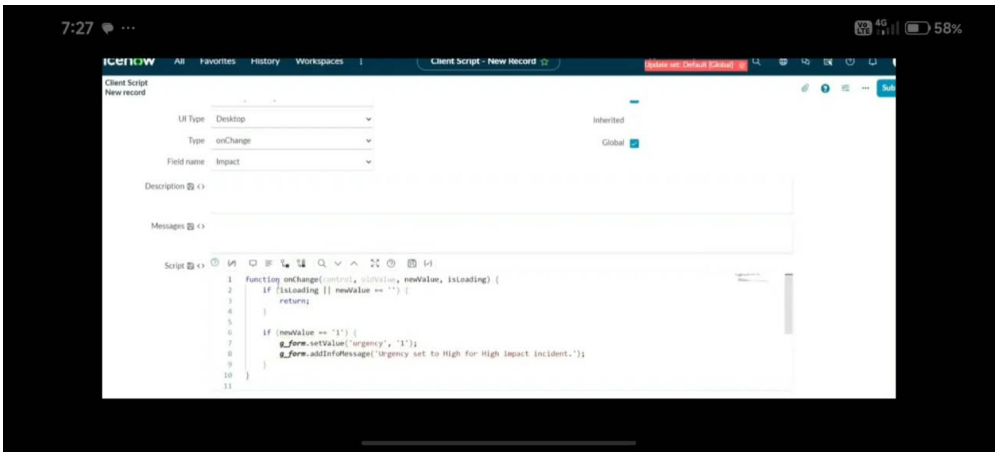
- Record the UI Policy name: “High Impact Control”, and the Incident table.
- Document the condition: Impact is 1 - High.
- Record the UI Policy Action that makes Assignment group mandatory when the condition is true.
- Include a screenshot of the condition and field-action settings.

3. Documenting Client Script Configuration

- Record the Client Script name, table, UI type, script type, and field name.
- For the shown onChange script, document that it checks the Impact value.
- When Impact is High (value “1”), the script sets Urgency to High (value “1”) and displays an informational message.
- Include the script screenshot and explain the expected form behaviour.

4. Test Results and Conclusion

- Test with High and non-High Impact values and record the observed Urgency value.
- Verify the Assignment group requirement and ensure the form behaves as expected.
- Attach screenshots as evidence and document any issues and corrections.
- The completed report provides a reference for maintenance, troubleshooting, and future enhancements.



IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Phase 8: Project Demonstration Phase

Objective: Demonstrate the completed ServiceNow Incident Client Script and UI Policy by showing how the form responds to a high-impact Incident and validates required assignment information.

1. Demonstration Setup

- Open the Incident form in the ServiceNow development or test instance.
- Create a new Incident record and enter the required caller and short description details.
- Set Impact to “1 - High” to trigger the configured rules.

2. Client Script Demonstration

- The onChange Client Script checks the Impact field when its value changes.
- When Impact is High, the script sets Urgency to “1 - High”.
- An informational message confirms that Urgency has been set for a high-impact Incident.

3. UI Policy Demonstration

- When Impact is High, the UI Policy makes Assignment group mandatory.
- The form displays the mandatory indicator beside Assignment group.
- The field remains visible and editable, consistent with the UI Policy Action settings.

4. Submission Validation and Result

- Attempt to submit the Incident without the required assignment information and observe the validation message.
- Enter the required Assignment group and submit again to verify the expected behaviour.
- Capture the form and validation messages as evidence of the demonstration.
- The screenshot shows the High Impact condition, High Urgency, mandatory Assignment group, and the validation message.

